Introduction

Read all instructions and questions carefully. Write your name (matching your registered name on Gradescope) and NetID (i.e. the first part of your NYU email address) and Section Name at the top of the paper.

Do not use any notes or other written materials. Do not use a phone, calculator, or other electronic device. Do not wear a smart watch. Do not fold, tear, or detach the exam paper. During the test, conversation between students is prohibited. If you have a question about the content of the exam, please ask the instructor.

Your test will be graded solely on the basis of what you have written. Answer only the questions asked. Please write legibly. In case of ambiguity, please circle your final answer. You may use the back of the page as scrap paper, but you may not write your answer on the back.

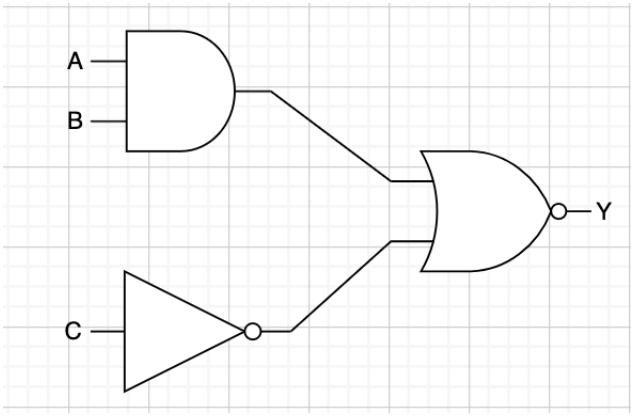
Handouts

Please note the handouts that you can use in answering questions:

- E15 Assembly Instructions Set and Equivalent Machine Codes
- E20 Assembly Instructions Set and Equivalent Machine Codes

Questions

1. Consider the following circuit.



(a) Write a combinatorial Verilog module named `testmodule` for the above circuit. Its input ports are A,B, and C. Its output port, Y, is the corresponding output of the above circuit diagram. Use only Verilog's low-level logical operations (operators `|`, `&`, `^`, $\sim$) in your answer.

Your module will start like this:

```
module testmodule(A, B, C,Y);
```

Be sure to declare your ports as `input` or `output`; declare any additional `wire`s you may need; and use `assign` statements to connect ports to gates. Your solution must be combinatorial. You may not use `always`, `reg`, or sequential assignment.

(b) Complete the truth table (in next page) for the above circuit.

A	B	C	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

2.

(a) Complete the following C function, which will return a value equal to its parameter, with the fourth and third least significant bits toggled, i.e. if the fourth least significant bit is on, turn it off, and if it is off, turn it on and if the thrid least significant bit is on, turn it off and if it is off, turn it on. For example, `flipbits_fourth_third(39)` should return 43. For another example, `flipbits_fourth_third(51)` should return 63.

```
unsigned int flipbits_fourth_third(unsigned int x) {
    return YOUR CODE HERE;
}
```

Your solution should be one line. Write your answer in the following text box. Use *only* the C/C++ bit-twiddling operators `&`, `|`, `^`, and $\sim$, as well as the shift operators `<<` and `>>`.

(b) Assume that the following operations are calculated using **7-bit 2's complement numbers**. Give the resulting sum in decimal. Take into account the effects of possible overflow and **show your work**.

i. $39 + 27$

ii. $-25 + -50$

3. Consider the following fragments of unrelated E15 assembly language programs (a and b) and E15 machine language code (c):

(a)
```
myROM[8] = { movi, RXX, Rg3, 4'b1101 };
myROM[9] = { movi, RXX, Rg3, 4'b1000 };
myROM[10] = { cmp, Rg3, Rg3, 4'b0000 };
myROM[11] = { jz, RXX, RXX,  4'b1111 };
```
What is the address of the instruction that will be executed immediately after the instruction at address 11? Give your answer in decimal. If the address of the next instruction cannot be determined from the information given, explain why.

(b)
```
myROM[8] = { movi, RXX, Rg0, 4'b0100 };
myROM[9] = { cmp,  Rg0, Rg2, 4'b0000 };
myROM[10] = { jnz,  RXX, RXX, 4'b0001 };
```
What is the address of the instruction that will be executed immediately after the instruction at address 10? Give your answer in decimal. If the address of the next instruction cannot be determined from the information given, explain why.

(c) For the following 12-bit number, interpret the number as an E15 assembly instruction. Write the instruction, including its opcode and arguments, and state in prose what it does, and specifically describe the modifications to any relevant registers, including the zero flag and program counter. If the assembly instruction ignores any of its register operands, use the symbol `RXX` to indicate so.

```
1100 00 01 0101
```

4. Consider each of the following complete E20 assembly language programs.

(a)
```
        movi $1, 1
        movi $2, 1
        movi $3, 5
label1:
        addi $1, $1, 1
        add  $2, $2, $2

        jeq  $1, $3, label2
        j    label1
label2:
        nop
        halt
```

What is the final value of `$1`? Give your answer in decimal. If the value cannot be determined, state why.

What is the final value of `$2`? Give your answer in decimal. If the value cannot be determined, state why.

(b)
```
        movi  $1,  label1
        lw    $2,  label3($0)
        and   $3,  $1,  $2
        or    $4,  $1,  $2
        halt
label1:
        .fill 29
label3:
        .fill 13
```

What is the final value of `$1`? Give your answer in decimal. If the value cannot be determined, state why.

What is the final value of `$3`? Give your answer in decimal. If the value cannot be determined, state why.

What is the final value of `$4`? Give your answer in decimal. If the value cannot be determined, state why.

5. Your E20's memory unit has been initialized with two arrays. The first array begins at the address identified by the label `first` and the second array begins at the address identified by the label `second`. Each array contains a sequence of non-zero integers, followed by a single memory cell containing zero. Both arrays are the same length.

   Write a complete E20 assembly language program to subtract the pairwise elements of each array into a third array beginning at the address identified by the label `third`. In other words, your program should provide the following behavior (expressed in C++ syntax):

```
third[0] = first[0] - second[0];
third[1] = first[1] - second[1];
third[2] = first[2] - second[2];
.... continue for length of arrays
```

   Your program should work for arrays of arbitrary length. You do not have to store a zero value at the end of the `third` array. After subtracking the arrays, your program should halt. Use correct E20 syntax. Write legibly, in neat rows. You may assume that the arrays' contents and the labels `first`, `second`, and `third` have been defined for you, but you must provide all other elements of the program. Your solution must use a loop.